St. Francis Institute of Technology, Mumbai-400 103

Department of Information Technology

A.Y. 2025-2026

Class: BE Honours, Semester: VII

Subject: **AI-ML in Healthcare Lab**

Experiment – 6: To Predict disease risk from patient data. (Breast Cancer prediction using SVM)

1. **Aim:** To predict disease risk from patient data.
2. **Objectives:** After study of this experiment, the student will be able to
   - Understand Breast Cancer Dataset
   - Understand Support Vector Machine algorithm
3. **Outcomes:** After study of this experiment, the student will be able to
   - Understand Breast Cancer Prediction using SVM..
4. **Prerequisite:** Fundamentals of Python Programming and Machine Learning.
5. **Requirements:** Python Installation, Personal Computer, Windows operating system, Internet Connection, Microsoft Word.
6. **Pre-Experiment Exercise:**

Brief Theory:

Matplotlib

Matplotlib is one of the most widely used Python libraries for data visualization. It provides a flexible way to create a wide range of static, animated, and interactive plots. It is particularly useful for generating publication-quality charts and customizing every element of a graph.

Key Concepts:

- **Figure**:
  The entire window or canvas on which plots are drawn. Created using plt.figure().
- **Axes**:
  The actual plotting area where data points are displayed. One figure can contain multiple axes. Created using fig.add_subplot() or plt.subplots().
- **Plotting Functions**:
  - plt.plot(): Line plots
  - plt.bar(): Bar charts
  - plt.hist(): Histograms
  - plt.scatter(): Scatter plots
  - plt.pie(): Pie charts
- **Labels and Titles**:
  - plt.xlabel('X-axis Label')
  - plt.ylabel('Y-axis Label')
  - plt.title('Chart Title')
    These functions help in labeling axes and providing titles for better understanding.
- **Legends and Grids**:
  - plt.legend(): Displays labels for different data series.
  - plt.grid(True): Adds grid lines for readability.
- **Customization**:
  Matplotlib allows full control over color, style, size, font, line thickness, and marker styles.
  Example: plt.plot(x, y, color='red', linestyle='--', marker='o')
- **Subplots**:
  You can display multiple plots in one figure using plt.subplot(rows, cols, index) or plt.subplots().

- **Saving Figures**:
  Use plt.savefig('filename.png') to export plots as image files.

Seaborn

Seaborn is a statistical data visualization library built on top of Matplotlib. It provides a high-level interface for creating visually appealing and informative statistical graphics. Seaborn integrates closely with **Pandas DataFrames**, making it ideal for exploratory data analysis.

Key Concepts:

- **Integration with DataFrames**:
  Seaborn works directly with Pandas DataFrames and uses column names to map variables to axes, colors, or sizes.
- **Themes and Styles**:
  Seaborn offers attractive default themes to enhance visual aesthetics.
  Common themes: darkgrid, whitegrid, dark, white, and ticks
  Example: sns.set_style('whitegrid')
- **High-Level Plotting Functions**:
  - sns.barplot(): Shows the mean value of a variable with confidence intervals
  - sns.histplot(): Creates histograms
  - sns.boxplot(): Visualizes data distribution and outliers
  - sns.violinplot(): Combines boxplot and kernel density estimation
  - sns.heatmap(): Displays matrix-like data as a color-coded grid
  - sns.pairplot(): Plots pairwise relationships between variables
  - sns.lmplot(): Fits and plots regression lines
- **Statistical Support**:
  Seaborn can automatically calculate and display confidence intervals and regression lines, helping visualize trends.
- **Customization**:
  Seaborn supports color palettes such as deep, muted, pastel, and allows hue-based differentiation.
- **FacetGrid and Subplots**:
  Seaborn provides tools like FacetGrid and catplot() to create multiple subplots based on different categories.

Comparison between Matplotlib and Seaborn

Feature	Matplotlib	Seaborn
Level	Low-level plotting library	High-level interface built on Matplotlib
Ease of Use	More code needed for complex visuals	Fewer lines of code for beautiful plots
Customization	Full control over every plot element	Limited but aesthetically pleasing defaults
Integration	Works with arrays and lists	Works seamlessly with Pandas DataFrames
Statistical Plots	Needs manual computation	Built-in statistical summaries and plots
Default Styles	Basic by default	Attractive and modern by default

7. Laboratory Exercise

A. Procedure:

```
from sklearn import datasets
```

NAME: Tanmay Bhatkar Class: BE INFT A SR no/ Roll no=>8

```
cancer = datasets.load_breast_cancer()
print("features",cancer.feature_names)
```

```
features ['mean radius' 'mean texture' 'mean perimeter' 'mean area'
 'mean smoothness' 'mean compactness' 'mean concavity'
 'mean concave points' 'mean symmetry' 'mean fractal dimension'
 'radius error' 'texture error' 'perimeter error' 'area error'
 'smoothness error' 'compactness error' 'concavity error'
 'concave points error' 'symmetry error' 'fractal dimension error'
 'worst radius' 'worst texture' 'worst perimeter' 'worst area'
 'worst smoothness' 'worst compactness' 'worst concavity'
 'worst concave points' 'worst symmetry' 'worst fractal dimension']
```

```
print("labels",cancer.target_names)
```

```
labels ['malignant' 'benign']
```

```
cancer.data.shape
```

```
(569, 30)
```

```
print(cancer.target)
```

```
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0
 1 0 0 0 0 0 0 0 0 1 0 1 1 1 1 1 0 0 1 0 0 1 1 1 1 0 1 0 0 1 1 1 1 0 1 0 0
 1 0 1 0 0 1 1 1 0 0 1 0 0 0 1 1 1 0 1 1 0 0 1 1 1 0 0 1 1 1 1 0 1 1 0 1 1
 1 1 1 1 1 1 0 0 0 1 0 0 1 1 1 0 0 1 0 1 0 0 1 0 0 1 1 0 1 1 0 1 1 1 1 1 0 1
 1 1 1 1 1 1 1 1 0 1 1 1 1 0 0 1 0 1 1 0 0 1 1 0 0 1 1 1 1 1 0 1 1 0 0 0 1 0
 1 0 1 1 1 0 1 1 0 0 1 0 0 0 0 1 0 0 0 1 0 1 0 1 1 0 1 0 0 0 0 1 1 0 0 1 1
 1 0 1 1 1 1 1 1 0 0 1 1 0 1 1 0 0 1 0 1 1 1 1 0 1 1 1 1 1 0 1 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 1 1 1 1 1 1 1 0 1 0 1 1 0 1 1 0 1 0 0 1 1 1 1 1 1 1 1 1 1 1 1
 1 0 1 1 0 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 1 1 1 0 1 0 1 1 1 1 1 0 0 0 1 1
 1 1 0 1 0 1 0 1 1 1 0 1 1 1 1 1 1 1 1 0 0 0 1 1 1 1 1 1 1 1 1 1 1 1 0 0 1 0 0
 0 1 0 0 1 1 1 1 1 0 1 1 1 1 1 0 1 1 1 0 1 1 0 0 1 1 1 1 1 1 1 0 1 1 1 1 1 1
 1 0 1 1 1 1 1 0 1 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 0 1 0 0 1 0 1 1 1 1 1 0 1 1
 0 1 0 1 1 0 1 0 1 1 1 1 1 1 1 1 1 1 0 0 1 1 1 1 1 1 0 1 1 1 1 1 1 1 1 1 1 0 1
 1 1 1 1 1 1 0 1 0 1 1 0 1 1 1 1 1 1 0 0 1 0 1 0 1 1 1 1 1 0 1 1 0 1 0 1 0 0
 1 1 1 0 1 1 1 1 1 1 1 1 1 1 1 1 0 1 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 0 0 0 0 0 0 1]
```

```
from sklearn.model_selection import train_test_split

X_train,X_test , y_train,y_test =
train_test_split(cancer.data,cancer.target,test_size =
0.3,random_state = 42)

from sklearn import svm
clf =svm.SVC(kernel='linear')
clf.fit(X_train,y_train)
y_pred=clf.predict(X_test)
from sklearn import metrics
print ("accuracy", metrics.accuracy_score(y_test,y_pred))
print ("precision",metrics.precision_score(y_test,y_pred))
print ("recall",metrics.recall_score(y_test,y_pred))
```

```
accuracy 0.9649122807017544
precision 0.9636363636363636
recall 0.9814814814814815
```

NAME: Tanmay Bhatkar Class: BE INFT A SR no/ Roll no=>8

8. Post-Experiments Exercise

A. Extended Theory:

Different Classification Algorithms

Classification is a **supervised machine learning** technique used to categorize data into predefined classes or labels. It learns from a labeled dataset (training data) and then predicts the class of new, unseen data.

Classification algorithms can be **binary** (two classes) or **multi-class** (more than two classes). Below are the **main types of classification algorithms** used in machine learning:

1. Logistic Regression

- **Type**: Linear classifier
- **Concept**: Despite its name, it is used for **classification**, not regression. It models the probability that a given input belongs to a particular class using the **sigmoid function**.
- **Output**: Probability between 0 and 1.
- **Use Case**: Spam detection (spam or not), disease prediction (yes or no).
- **Advantages**: Simple, interpretable, works well with linearly separable data.

2. K-Nearest Neighbors (KNN)

- **Type**: Instance-based, non-parametric algorithm
- **Concept**: Classifies a new data point based on the **majority class** among its **k nearest neighbors** in the feature space.
- **Distance Metrics**: Euclidean, Manhattan, or Minkowski distance.
- **Use Case**: Pattern recognition, recommendation systems.
- **Advantages**: Simple and effective; no training phase.
- **Limitations**: Slow for large datasets; sensitive to noise and irrelevant features.

3. Decision Tree Classifier

- **Type**: Tree-based model
- **Concept**: Splits data into branches based on **feature values** using metrics like **Gini index** or **Information Gain** until a decision is reached.
- **Output**: A tree-like structure where leaves represent class labels.
- **Use Case**: Credit scoring, medical diagnosis.
- **Advantages**: Easy to interpret; handles both numerical and categorical data.
- **Limitations**: Prone to **overfitting**; small changes in data can alter the tree structure.

4. Random Forest Classifier

- **Type**: Ensemble learning (combination of multiple models)
- **Concept**: Builds **multiple decision trees** and combines their outputs (via majority voting) to improve accuracy and reduce overfitting.
- **Use Case**: Fraud detection, sentiment analysis.
- **Advantages**: Robust, handles missing values, less prone to overfitting.
- **Limitations**: Less interpretable, more computationally expensive.

5. Support Vector Machine (SVM)

- **Type**: Margin-based classifier
- **Concept**: Finds the **optimal hyperplane** that separates classes with the **maximum margin**.
- **Kernels**: Linear, Polynomial, RBF (Radial Basis Function) for non-linear data.
- **Use Case**: Image classification, text categorization.
- **Advantages**: Effective in high-dimensional spaces.
- **Limitations**: Slow for large datasets; needs careful kernel selection.

6. Naive Bayes Classifier

- **Type**: Probabilistic classifier
- **Concept**: Based on **Bayes' Theorem** and the assumption that features are **independent**.
- **Formula**:

$$P(Class|Features) = \frac{P(Features|Class) \times P(Class)}{P(Features)}$$

  Types: Gaussian, Multinomial, and Bernoulli Naive Bayes.
- **Use Case**: Spam filtering, sentiment analysis.
- **Advantages**: Fast, works well with small datasets, good for text data.
- **Limitations**: Assumes independence between features.

7. Gradient Boosting Classifier

- **Type**: Ensemble learning
- **Concept**: Builds models **sequentially**, where each new model corrects the errors of the previous one.
- **Popular Variants**:
  - **AdaBoost (Adaptive Boosting)**
  - **XGBoost (Extreme Gradient Boosting)**
  - **LightGBM**
  - **CatBoost**
- **Use Case**: Kaggle competitions, complex classification tasks.
- **Advantages**: High accuracy, handles missing data.
- **Limitations**: Can overfit; requires careful tuning.

8. Neural Network Classifiers

- **Type**: Deep learning-based
- **Concept**: Mimics human brain structure using layers of interconnected neurons. Each layer extracts features from data.
- **Variants**:
  - **Feedforward Neural Networks (FNN)**
  - **Convolutional Neural Networks (CNN)** for image data
  - **Recurrent Neural Networks (RNN)** for sequential data
- **Use Case**: Image recognition, voice recognition, complex pattern detection.
- **Advantages**: Can learn complex non-linear relationships.
- **Limitations**: Requires large datasets and high computation.

9. Convolutional Neural Networks (CNN)

- **Type**: Deep learning, non-linear classifier
- **Concept**:
  Designed for data with spatial structures (like images).
  Uses **convolutional layers** to extract features automatically, **pooling layers** for dimensionality reduction, and **fully connected layers** for classification.
- **Architecture Components**:
  - **Convolutional Layer**: Feature extraction
  - **Activation (ReLU)**: Non-linearity
  - **Pooling Layer**: Reduces dimensions
  - **Fully Connected Layer**: Classification
  - **Softmax Output**: Gives class probabilities
- **Use Cases**: Image classification, object detection, medical imaging
- **Advantages**: High accuracy, automatic feature learning
- **Limitations**: Requires large datasets and GPUs

10. Recurrent Neural Networks (RNN)

- **Type**: Deep learning, sequential data classifier

- **Concept**:
  Designed for **time-series or sequential data**, where past information influences the current output.
  Includes variants like **LSTM (Long Short-Term Memory)** and **GRU (Gated Recurrent Unit)**.
- **Use Cases**: Speech recognition, sentiment analysis, stock prediction
- **Advantages**: Handles sequential dependencies
- **Limitations**: Training complexity, vanishing gradient problem

11. Quadratic Discriminant Analysis (QDA) and Linear Discriminant Analysis (LDA)

- **Concept**: Based on modeling class distributions using Gaussian assumptions.
- **LDA** assumes equal covariance; **QDA** allows different covariance matrices for each class.
- **Use Case**: Pattern recognition, classification with continuous variables.
- **Advantages**: Works well with normally distributed features.
- **Limitations**: Sensitive to non-Gaussian distributions.

12. Ensemble Voting Classifier

- **Concept**: Combines predictions from multiple models using:
  - **Hard Voting**: Majority class
  - **Soft Voting**: Weighted average of probabilities
- **Use Case**: Improves model stability and accuracy.
- **Advantage**: Reduces model bias and variance.
- **Limitation**: More computational resources required.

B. Questions:

Which is better SVM or CNN and when?

C. Conclusion:

Write the significance of the topic studied in the experiment.

D. References:

https://www.analyticsvidhya.com/blog/2021/06/build-an-image-classifier-with-svm/
https://iopscience.iop.org/article/10.1088/1755-1315/357/1/012035/pdf